

ASSIGNMENT-2.3

2303A52429

BATCH-32

TASK-1

CODE

```
2303A52429
VALLALA VYSHNAVI
BATCH-32

# 1. Create a Sample Text File
file_content = """This is a sample text file. This file contains some sample text to demonstrate word counting. Sample counting is fun!"""

with open('sample.txt', 'w') as file:
    file.write(file_content)

print("Sample file 'sample.txt' created successfully.")

# 2. Read the File and Initialize the Dictionary
word_frequency = {}

try:
    with open('sample.txt', 'r') as file:
        text = file.read()
        print(f"File 'sample.txt' read successfully. Content:\n{text}\n")
except FileNotFoundError:
    print("Error: The file 'sample.txt' was not found.")
    text = ""

# 3. Process the Text and Count Word Frequencies
if text:
    # Convert to lowercase
    text = text.lower()

    # Remove punctuation (simple approach, can be more robust)
    import re
    text = re.sub(r'[!@#$%^&*()-+=~`~\.\,;:\'"]', '', text)

    # Split into words
    words = text.split()

    # Count frequencies
    for word in words:
        if word:
            word_frequency[word] = word_frequency.get(word, 0) + 1

    print("Word frequency counting complete.")
else:
    print("No text to process.")

# 4. Display the Word Frequencies
print("\n--- Word Frequencies ---")
# Sort the dictionary by frequency (optional)
sorted_word_frequency = sorted(word_frequency.items(), key=lambda item: item[1], reverse=True)

for word, count in sorted_word_frequency:
    print(f'{word}: {count}')
```

```
# 3. Process the Text and Count Word Frequencies
if text:
    # Convert to lowercase
    text = text.lower()

    # Remove punctuation (simple approach, can be more robust)
    import re
    text = re.sub(r'[!@#$%^&*()-+=~`~\.\,;:\'"]', '', text)

    # Split into words
    words = text.split()

    # Count frequencies
    for word in words:
        if word:
            word_frequency[word] = word_frequency.get(word, 0) + 1

    print("Word frequency counting complete.")
else:
    print("No text to process.")

# 4. Display the Word Frequencies
print("\n--- Word Frequencies ---")
# Sort the dictionary by frequency (optional)
sorted_word_frequency = sorted(word_frequency.items(), key=lambda item: item[1], reverse=True)

for word, count in sorted_word_frequency:
    print(f'{word}: {count}')
```

```

*** Sample file 'sample.txt' created successfully.
File 'sample.txt' read successfully. Content:
This is a sample text file. This file contains some sample text to demonstrate word counting. Sample counting is fun!

Word frequency counting complete.

--- Word Frequencies ---
'sample': 3
'this': 2
'is': 2
'text': 2
'file': 2
'counting': 2
'a': 1
'contains': 1
'some': 1
'to': 1
'demonstrate': 1
'word': 1
'fun': 1

```

EXPLANATION

Explanation of the Code

1. Create a Sample Text File

This section creates a file named `sample.txt` with some predefined text. This is purely for demonstration purposes so that the word counting program has a file to work with immediately.

2. Read the File and Initialize the Dictionary

- An empty dictionary named `word_frequency` is initialized. This dictionary will store each unique word found in the text as a key and its occurrence count as the corresponding value.
- The `sample.txt` file is opened in read mode (`'r'`).
- The entire content of the file is read into the `text` variable.
- A `try-except` block is used to gracefully handle cases where the file might not be found.

3. Process the Text and Count Word Frequencies

This is the core logic for counting word frequencies:

- **Convert to Lowercase:** `text.lower()` converts all characters in the text to lowercase. This ensures that words like "The" and "the" are treated as the same word, preventing them from being counted separately.
- **Remove Punctuation:** `re.sub(r'[^\w-9\s]', '', text)` uses a regular expression to remove any characters that are not lowercase letters, numbers, or whitespace. This effectively cleans the text by stripping out punctuation marks like periods, commas, and exclamation points.
- **Split into Words:** `text.split()` breaks the cleaned text into a list of individual words, using whitespace as the delimiter.
- **Count Frequencies:** The code then iterates through each `word` in the `words` list.
 - For each `word`, it checks if the word is not empty (to handle multiple spaces). If valid, it updates its count in the `word_frequency` dictionary.

3. Process the Text and Count Word Frequencies

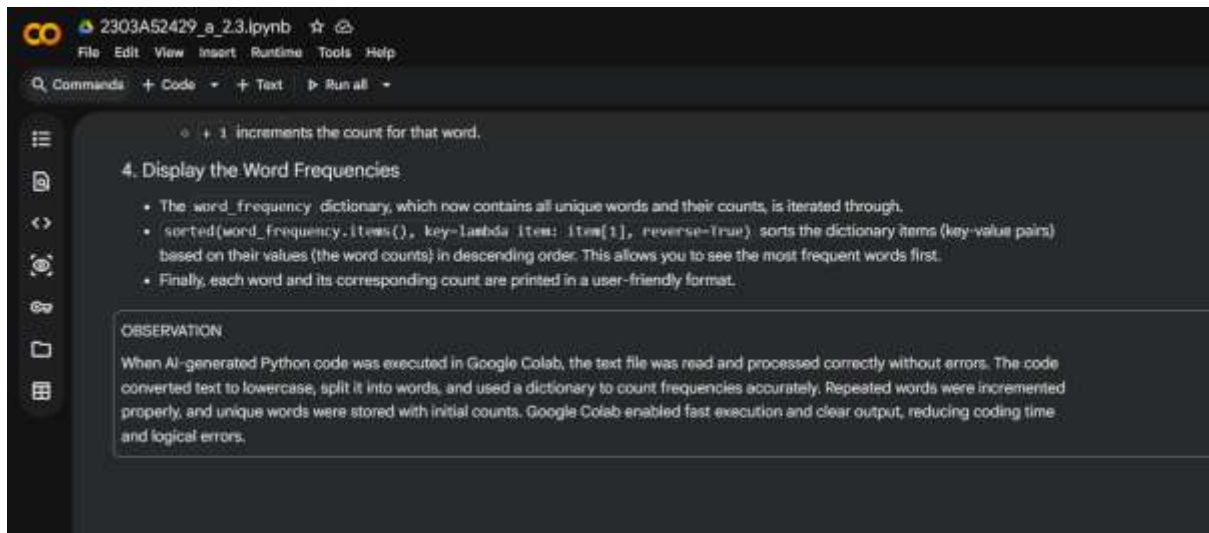
This is the core logic for counting word frequencies:

- **Convert to Lowercase:** `text.lower()` converts all characters in the text to lowercase. This ensures that words like "The" and "the" are treated as the same word, preventing them from being counted separately.
- **Remove Punctuation:** `re.sub(r'[^\w-9\s]', '', text)` uses a regular expression to remove any characters that are not lowercase letters, numbers, or whitespace. This effectively cleans the text by stripping out punctuation marks like periods, commas, and exclamation points.
- **Split into Words:** `text.split()` breaks the cleaned text into a list of individual words, using whitespace as the delimiter.
- **Count Frequencies:** The code then iterates through each `word` in the `words` list.
 - For each `word`, it checks if the word is not empty (to handle multiple spaces). If valid, it updates its count in the `word_frequency` dictionary.
 - `word_frequency.get(word, 0)` safely retrieves the current count of the `word`. If the word is not yet in the dictionary, it defaults to `0`.
 - `+ 1` increments the count for that word.

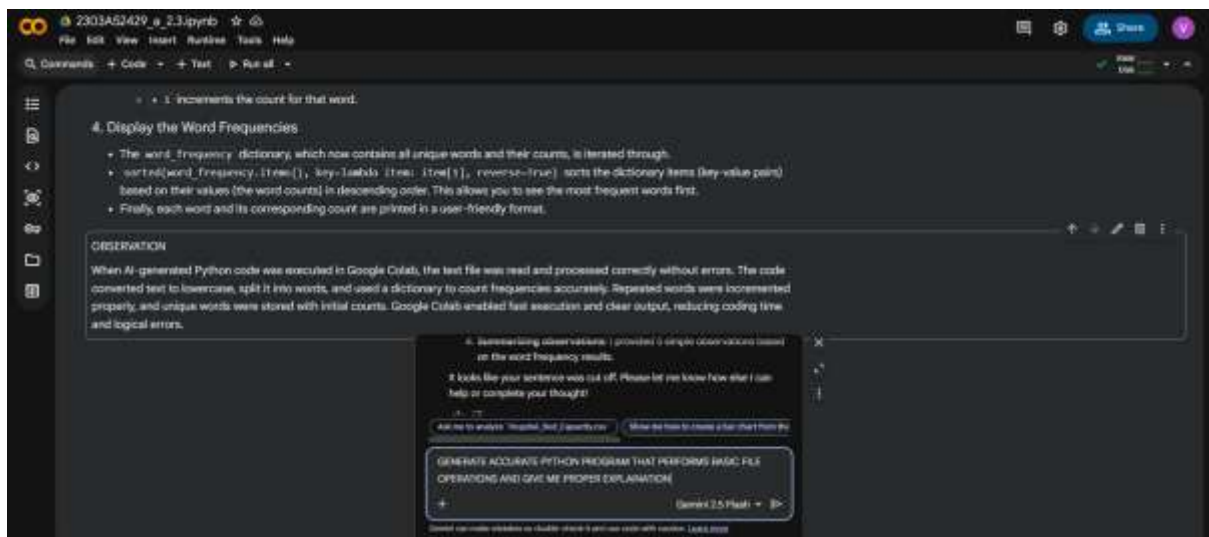
4. Display the Word Frequencies

- The `word_frequency` dictionary, which now contains all unique words and their counts, is iterated through.
- `sorted(word_frequency.items(), key=lambda item: item[1], reverse=True)` sorts the dictionary items (key-value pairs) based on their values (the word counts) in descending order. This allows you to see the most frequent words first.
- Finally, each word and its corresponding count are printed in a user-friendly format.

OBSERVATION



TASK-2



2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Python Program for Basic File Operations

This program will demonstrate how to perform common file operations such as creating, writing, reading, appending to, and deleting a file.

[10] ✓ On

1. Create and Write to a New File
file_name = 'my_example_file.txt'
content_to_write = "Hello, this is the first line.\nThis is the second line.\n"

try:
 with open(file_name, 'w') as file:
 file.write(content_to_write)
 print(f"Successfully created and wrote to '{file_name}'.")
except IOError as e:
 print(f"Error writing to file: {e}")

Successfully created and wrote to 'my_example_file.txt'.

[11] ✓ On

2. Read from the File

try:
 with open(file_name, 'r') as file:
 read_content = file.read()
 print(f"\nContent of '{file_name}':\n{read_content}")
except FileNotFoundError:
 print(f"Error: The file '{file_name}' was not found for reading.")
except IOError as e:
 print(f"Error reading from file: {e}")

Content of 'my_example_file.txt':
Hello, this is the first line.
This is the second line.

2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

3. Append to the File
additional_content = "This line is appended later.\nAnother appended line.\n"

try:
 with open(file_name, 'a') as file:
 file.write(additional_content)
 print(f"Successfully appended to '{file_name}'.")
except IOError as e:
 print(f"Error appending to file: {e}")

*** Successfully appended to 'my_example_file.txt'.

[11] ✓ On

4. Read the File Again (to see appended content)

try:
 with open(file_name, 'r') as file:
 updated_content = file.read()
 print(f"\nUpdated content of '{file_name}' after appending:\n{updated_content}")
except FileNotFoundError:
 print(f"Error: The file '{file_name}' was not found for reading.")
except IOError as e:
 print(f"Error reading from file: {e}")

Updated content of 'my_example_file.txt' after appending:
Hello, this is the first line.
This is the second line.
This line is appended later.
Another appended line.

2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[11] ✓ OK

```
with open(file_name, 'r') as file:
    updated_content = file.read()
    print(f'\nUpdated content of '{file_name}' after appending:\n{updated_content}')
except FileNotFoundError:
    print(f'Error: The file '{file_name}' was not found for reading.')
except IOError as e:
    print(f'Error reading from file: {e}')
```

Updated content of 'my_example_file.txt' after appending:
Hello, this is the first line.
This is the second line.
This line is appended later.
Another appended line.

[12] ✓ OK

```
# 5. Delete the File
import os

try:
    os.remove(file_name)
    print(f'\nSuccessfully deleted '{file_name}'.')
except FileNotFoundError:
    print(f'Error: Cannot delete '{file_name}' because it was not found.')
except OSError as e:
    print(f'Error deleting file: {e}')
```

...
Successfully deleted 'my_example_file.txt'.

2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Explanation of the Python Program for Basic File Operations

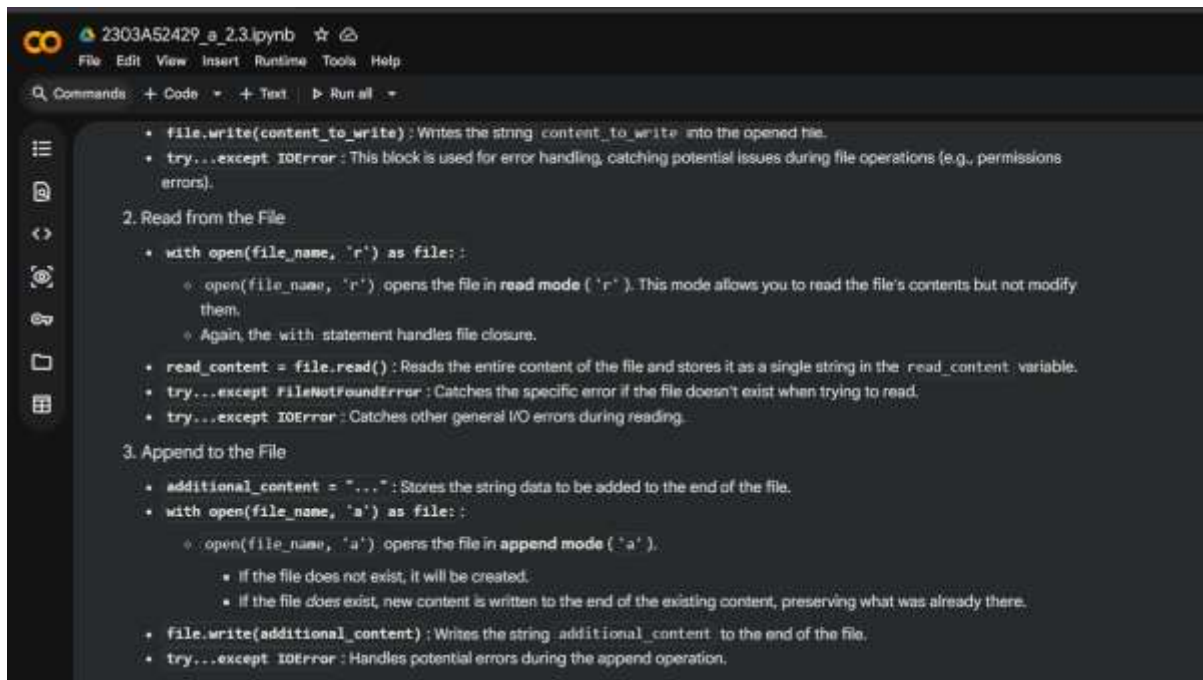
This program demonstrates five fundamental file operations in Python: creating/writing, reading, appending, and deleting.

1. Create and Write to a New File

- `file_name = 'my_example_file.txt'`: Defines the name of the file we'll be working with.
- `content_to_write = "..."`: Stores the string data that will be written into the file.
- `with open(file_name, 'w') as file:`:
 - `open(file_name, 'w')` opens the file in **write mode** ('w').
 - If the file does not exist, it will be created.
 - If the file *does* exist, its content will be **truncated** (erased) before new content is written.
 - The `with` statement ensures that the file is properly closed automatically, even if errors occur.
 - `as file` assigns the opened file object to the variable `file`.
- `file.write(content_to_write)`: Writes the string `content_to_write` into the opened file.
- `try...except IOError`: This block is used for error handling, catching potential issues during file operations (e.g., permissions errors).

2. Read from the File

- `with open(file_name, 'r') as file:`:
 - `open(file_name, 'r')` opens the file in **read mode** ('r'). This mode allows you to read the file's contents but not modify them.
 - Again, the `with` statement handles file closure.
- `read_content = file.read()`: Reads the entire content of the file and stores it as a single string in the `read_content` variable.



2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

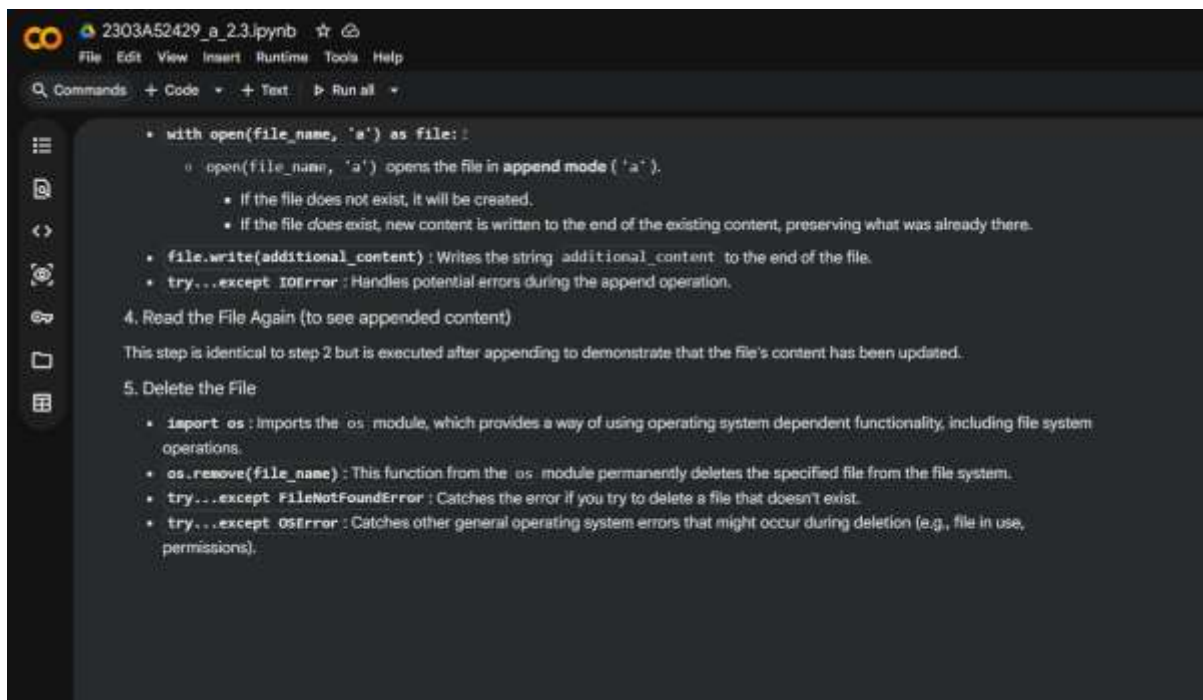
- `file.write(content_to_write)`: Writes the string `content_to_write` into the opened file.
- `try...except IOError`: This block is used for error handling, catching potential issues during file operations (e.g., permissions errors).

2. Read from the File

- `with open(file_name, 'r') as file:`
 - `open(file_name, 'r')` opens the file in **read mode** (`'r'`). This mode allows you to read the file's contents but not modify them.
 - Again, the `with` statement handles file closure.
- `read_content = file.read()`: Reads the entire content of the file and stores it as a single string in the `read_content` variable.
- `try...except FileNotFoundError`: Catches the specific error if the file doesn't exist when trying to read.
- `try...except IOError`: Catches other general I/O errors during reading.

3. Append to the File

- `additional_content = "..."`: Stores the string data to be added to the end of the file.
- `with open(file_name, 'a') as file:`
 - `open(file_name, 'a')` opens the file in **append mode** (`'a'`).
 - If the file does not exist, it will be created.
 - If the file does exist, new content is written to the end of the existing content, preserving what was already there.
- `file.write(additional_content)`: Writes the string `additional_content` to the end of the file.
- `try...except IOError`: Handles potential errors during the append operation.



2303A52429_a_2.3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

- `with open(file_name, 'a') as file:`
 - `open(file_name, 'a')` opens the file in **append mode** (`'a'`).
 - If the file does not exist, it will be created.
 - If the file does exist, new content is written to the end of the existing content, preserving what was already there.
- `file.write(additional_content)`: Writes the string `additional_content` to the end of the file.
- `try...except IOError`: Handles potential errors during the append operation.

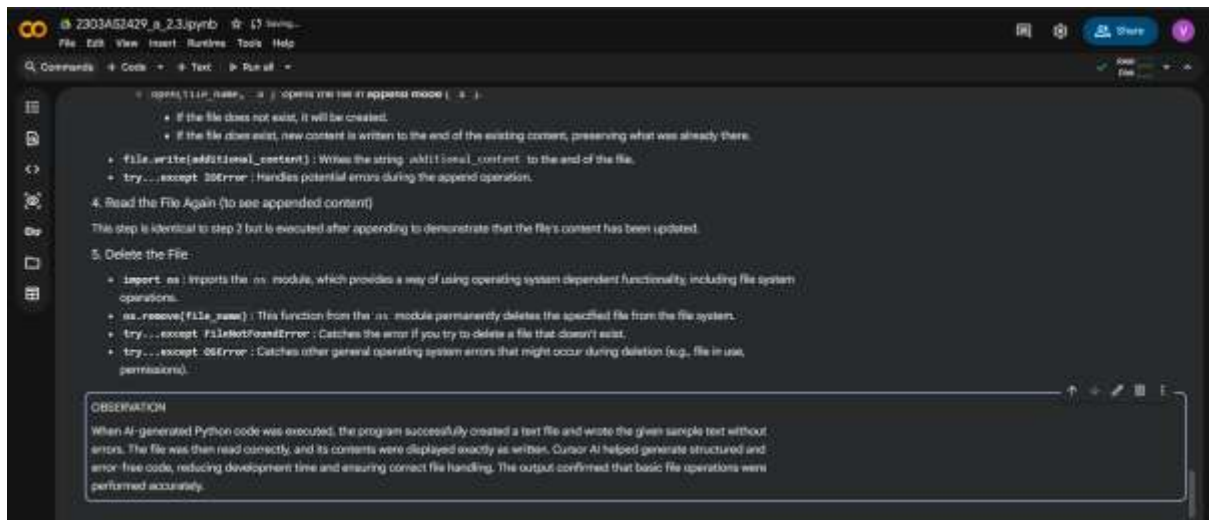
4. Read the File Again (to see appended content)

This step is identical to step 2 but is executed after appending to demonstrate that the file's content has been updated.

5. Delete the File

- `import os`: Imports the `os` module, which provides a way of using operating system dependent functionality, including file system operations.
- `os.remove(file_name)`: This function from the `os` module permanently deletes the specified file from the file system.
- `try...except FileNotFoundError`: Catches the error if you try to delete a file that doesn't exist.
- `try...except OSError`: Catches other general operating system errors that might occur during deletion (e.g., file in use, permissions).

OBSERVATION



TASK-3



```
2303A52429_a_2.3.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[11]
import pandas as pd

# Create a sample DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [24, 27, 22, 32, 29],
    'Score': [85.5, 91.2, 78.0, 88.9, 95.1],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']
}
df_sample = pd.DataFrame(data)

# Save the DataFrame to a CSV file
df_sample.to_csv('data.csv', index=False)

print("Sample 'data.csv' created successfully.")
print("--- Sample Data ---")
print(df_sample.head())

-- Sample 'data.csv' created successfully.
--- Sample Data ---
   Name  Age  Score  City
0  Alice   24   85.5  New York
1   Bob   27   91.2  Los Angeles
2 Charlie   22   78.0   Chicago
3  David   32   88.9   Houston
4   Eve   29   95.1   Phoenix
```

```
2303A52429_a_2.3.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[12]
import pandas as pd

try:
    # Read the CSV file into a DataFrame
    df = pd.read_csv('data.csv')
    print("\nCSV file 'data.csv' loaded successfully.")
    print("--- Loaded Data Head ---")
    print(df.head())

    # Select only numerical columns for statistical calculations
    numerical_cols = df.select_dtypes(include=['number']).columns

    if not numerical_cols.empty:
        print("\n--- Statistical Summary ---")
        # Calculate Mean
        mean_values = df[numerical_cols].mean()
        print("\nMean values:\n", mean_values)

        # Calculate Minimum
        min_values = df[numerical_cols].min()
        print("\nMinimum values:\n", min_values)

        # Calculate Maximum
        max_values = df[numerical_cols].max()
        print("\nMaximum values:\n", max_values)

        # You can also get a summary using .describe()
        print("\n--- Full Descriptive Statistics ---")
        print(df[numerical_cols].describe())
    else:
        print("\nNo numerical columns found to calculate statistics.")
```



```

        print("\n--- Full Descriptive Statistics ---")
        print(df[numerical_cols].describe())
    else:
        print("\nNo numerical columns found to calculate statistics.")

except FileNotFoundError:
    print("Error: 'data.csv' not found. Please ensure the file is in the correct directory.")
except Exception as e:
    print(f"An error occurred: {e}")

```

CSV file 'data.csv' loaded successfully.

--- Loaded Data Head ---

```

...      Name  Age  Score      City
0    Alice   24   85.5    New York
1     Bob   27   91.2  Los Angeles
2  Charlie   22   78.0    Chicago
3   David   32   88.9    Houston
4     Eve   29   95.1    Phoenix

```

--- Statistical Summary ---

Mean values:

```

Age      26.80
Score    87.74
dtype: float64

```

Minimum values:

```

Age      22.0
Score    78.0
dtype: float64

```

Maximum values:

```

Age      32.0
Score    95.1
dtype: float64

```

--- Full Descriptive Statistics ---

```

              Age      Score
count  5.000000  5.000000
mean   26.800000  87.740000
std     3.962323   6.468617
min    22.000000  78.000000
25%    24.000000  85.500000
50%    27.000000  88.900000
75%    29.000000  91.200000
max    32.000000  95.100000

```

EXPLANATION

Explanation of the Python Program

This program is divided into two main parts: creating a sample CSV (for demonstration purposes) and then reading that CSV to calculate basic statistics.

1. Create a Sample CSV File

- `import pandas as pd` : Imports the pandas library, which is essential for working with DataFrames and CSV files.
- `data = {...}` : A Python dictionary is created to hold sample data. Each key represents a column name, and its value is a list of data for that column.
- `df_sample = pd.DataFrame(data)` : This line converts the dictionary into a pandas DataFrame. A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types, similar to a spreadsheet or SQL table.
- `df_sample.to_csv('data.csv', index=False)` : This writes the `df_sample` DataFrame to a CSV file named `data.csv`. `index=False` prevents pandas from writing the DataFrame index as a column in the CSV.

2. Read the CSV File and Calculate Statistics

- `import pandas as pd` : (Re-imported for clarity, but only needed once at the top of a script).
- `try...except` block: This is used for robust error handling.
 - `FileNotFoundError` : Catches the error if `data.csv` doesn't exist.
 - `Exception as e` : Catches any other general errors that might occur during file processing.
- `df = pd.read_csv('data.csv')` : This is the core step for reading the CSV. `pd.read_csv()` is a powerful function that reads a CSV file into a pandas DataFrame. By default, it infers data types and uses the first row as headers.
- `numerical_cols = df.select_dtypes(include=['number']).columns` : This line identifies all columns in the DataFrame that have a numerical data type (integers, floats). We only want to calculate mean, min, and max for numbers.
- `if not numerical_cols.empty` : Checks if there are any numerical columns to process. If not, it prints a message.
- `mean_values = df[numerical_cols].mean()` :

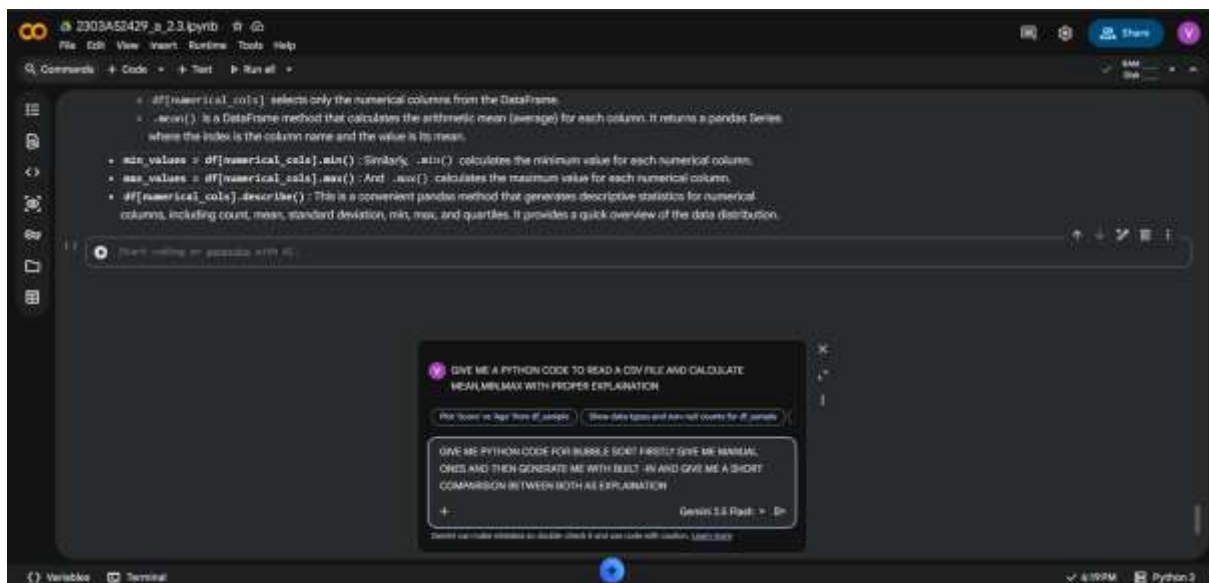
2. Read the CSV File and Calculate Statistics

- `import pandas as pd` : (Re-imported for clarity, but only needed once at the top of a script).
- `try...except` block: This is used for robust error handling.
 - `FileNotFoundError` : Catches the error if `data.csv` doesn't exist.
 - `Exception as e` : Catches any other general errors that might occur during file processing.
- `df = pd.read_csv('data.csv')` : This is the core step for reading the CSV. `pd.read_csv()` is a powerful function that reads a CSV file into a pandas DataFrame. By default, it infers data types and uses the first row as headers.
- `numerical_cols = df.select_dtypes(include=['number']).columns` : This line identifies all columns in the DataFrame that have a numerical data type (integers, floats). We only want to calculate mean, min, and max for numbers.
- `if not numerical_cols.empty` : Checks if there are any numerical columns to process. If not, it prints a message.
- `mean_values = df[numerical_cols].mean()` :
 - `df[numerical_cols]` selects only the numerical columns from the DataFrame.
 - `.mean()` is a DataFrame method that calculates the arithmetic mean (average) for each column. It returns a pandas Series where the index is the column name and the value is its mean.
- `min_values = df[numerical_cols].min()` : Similarly, `.min()` calculates the minimum value for each numerical column.
- `max_values = df[numerical_cols].max()` : And `.max()` calculates the maximum value for each numerical column.
- `df[numerical_cols].describe()` : This is a convenient pandas method that generates descriptive statistics for numerical columns, including count, mean, standard deviation, min, max, and quartiles. It provides a quick overview of the data distribution.

OBSERVATION

When AI-generated Python code was executed to analyze a CSV file, the program successfully read the dataset and extracted numerical values without errors. The code accurately calculated the mean, minimum, and maximum for the selected columns. Results were displayed clearly, making verification straightforward. Using AI assistance reduced logical mistakes and improved efficiency in basic data analysis tasks.

TASK-4



1. Manual Bubble Sort Implementation

This code implements the Bubble Sort algorithm from scratch. Bubble Sort repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.

```

[13] def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        # So, we only need to compare up to n-i-1
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
        return arr

# Example usage:
my_list = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {my_list}")
sorted_list_manual = bubble_sort(list(my_list)) # Use list(my_list) to avoid modifying original list directly for comparison
print(f"Sorted list (Manual Bubble Sort): {sorted_list_manual}")

my_strings = ['banana', 'apple', 'grape', 'orange']
print(f"Original strings list: {my_strings}")
sorted_strings_manual = bubble_sort(list(my_strings))
print(f"Sorted strings list (Manual Bubble Sort): {sorted_strings_manual}")

```

... Original list: [64, 34, 25, 12, 22, 11, 90]
 Sorted list (Manual Bubble Sort): [11, 12, 22, 25, 34, 64, 90]

Original strings list: ['banana', 'apple', 'grape', 'orange']
 Sorted strings list (Manual Bubble Sort): ['apple', 'banana', 'grape', 'orange']

2303A52429_p_2.3.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

2. Built-in Sorting Implementation

Python provides highly optimized built-in functions for sorting lists, namely `list.sort()` (which sorts the list in-place) and `sorted()` (which returns a new sorted list). These functions are typically implemented using algorithms like Timsort, which is a hybrid stable sorting algorithm, offering much better performance than Bubble Sort for most real-world datasets.

```
# Example usage with built-in sorted() function:
my_list_builtin = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {my_list_builtin}")
sorted_list_builtin = sorted(my_list_builtin)
print(f"Sorted list (Built-in sorted()): {sorted_list_builtin}")

my_strings_builtin = ['banana', 'apple', 'grape', 'orange']
print(f"Original strings list: {my_strings_builtin}")
sorted_strings_builtin = sorted(my_strings_builtin)
print(f"Sorted strings list (Built-in sorted()): {sorted_strings_builtin}")

# Example usage with list.sort() method (sorts in-place):
# my_list_inplace = [64, 34, 25, 12, 22, 11, 90]
# print(f"Original list for in-place sort: {my_list_inplace}")
# my_list_inplace.sort()
# print(f"Sorted list (list.sort() in-place): {my_list_inplace}")
```

Original list: [64, 34, 25, 12, 22, 11, 90]

Sorted list (Built-in sorted()): [11, 12, 22, 25, 34, 64, 90]

Original strings list: ['banana', 'apple', 'grape', 'orange']

Sorted strings list (Built-in sorted()): ['apple', 'banana', 'grape', 'orange']

Comparison between Manual Bubble Sort and Built-in Sorting

Feature	Manual Bubble Sort	Built-in Sorting (<code>sorted()</code> / <code>list.sort()</code>)
Algorithm	Simple comparison-based algorithm.	Hybrid algorithms (e.g., Timsort), highly optimized.
Time Complexity	$O(n^2)$ in worst and average cases.	$O(n \log n)$ in worst and average cases.
Efficiency	Very inefficient for large datasets.	Highly efficient and fast for most scenarios.
Readability	Easy to understand the logic.	Simple to use (single function/method call).
Use Cases	Educational purposes, small arrays.	Practically all real-world sorting tasks.

In summary, while Bubble Sort is excellent for understanding basic sorting concepts, Python's built-in sorting functions are vastly superior in performance and should always be preferred for actual applications due to their optimized implementations.

Both Bubble Sort and Python's built-in `sort()` correctly sorted the list. Bubble Sort took more time because it uses repeated comparisons and swaps. The built-in `sort()` executed faster with fewer lines of code. This shows built-in sorting is more efficient, while Bubble Sort is mainly for learning.